

INFORME COMPILADORES - PARTE 4

Alumna: Luciana Scheid

La Parte 4 del compilador implementa el Generador de Código que es la última fase del proceso de compilación. A partir del AST validado semánticamente, esta etapa produce archivos de bytecode Jasmin (.j) que la JVM puede ensamblar y ejecutar.

Clase principal: JasminCodeGenerator

Implementa la interfaz Visitor y recorre el AST generando instrucciones Jasmin para cada nodo.

Estructura general:

- classCode: StringBuilder que acumula el código de la clase actual.
- methodBody: StringBuilder para el cuerpo del método en generación.
- localVars de JasminVariableScope: mapeo de nombre para el índice de slot JVM y tipo.
- exprCode y exprType: pilas de strings para composición de expresiones.

labelCounter: contador para generar etiquetas únicas

Métodos auxiliares:

- jasminType: Convierte un nodo tipo AST a descriptor JVM.
- jasminTypeFromName: Convierte un nodo tipo AST pero desde un string.
- binaryArith: Genera iadd/isub/imul/ldiv visitando ambos operandos.
- binaryCompare: Genera comparaciones con saltos dejando 0 o 1 en la pila
- newLabel(): Produce etiquetas únicas L0, L1, etc
- lookupFieldType: Busca el tipo de un campo subiendo la jerarquía de herencia vía ProgramContainer

Clase auxiliar: JasminVariableScope

Gestiona el ámbito de variables locales de cada método usando dos HashMaps. Tiene los métodos insert(), lookupIndex() y lookupType() utilizados por el generador para emitir las instrucciones de carga y almacenamiento correctas.

Modo de ejecución:

- en el build.xml : se agregó targets para correr run-jasmin, se tiene que ejecutar después del run-main, ya que primero es necesario procesar los .j que se genera. El target run-compiled se encarga de ejecutar la clase compilada que está en la carpeta output

Nota: Asegurarse que la librería jasmin.jar esté cargada en las propiedades

Targets

```

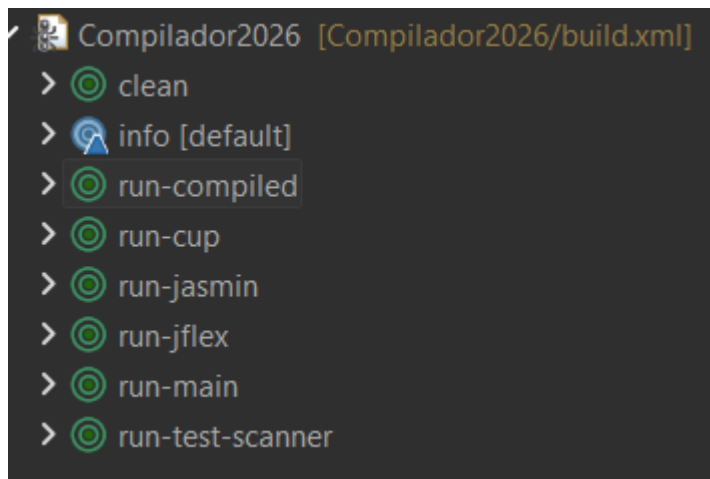
<target name="run-jasmin" depends="run-main">
    <apply executable="java" parallel="false">
        <arg value="-jar"/>
        <arg value="lib/jasmin.jar"/>
        <arg value="-d"/>
        <arg value="output"/>
        <srcfile/>
        <fileset dir="output" includes="*.j"/>
    </apply>
</target>

<target name="run-compiled" depends="run-jasmin">
    <java classname="Factorial" fork="true">
        <classpath>
            <pathelement location="output"/>
        </classpath>
    </java>
</target>

```

 output

-  BBS.class
-  BBS.j
-  BubbleSort.class
-  BubbleSort.j
-  Fac.class
-  Fac.j
-  Factorial.class
-  Factorial.j
-  QS.class
-  QS.j
-  QuickSort.class
-  QuickSort.j



Ejemplo: Factorial

Factorial.j

```
.class public Factorial
.super java/lang/Object
.method public <init>()V
    .limit stack 1
    .limit locals 1
    aload_0
    invokespecial java/lang/Object/<init>()V
    return
.end method
.method public static main([Ljava/lang/String;)V
    .limit stack 20
    .limit locals 1
    getstatic java/lang/System/out Ljava/io/PrintStream;
    new Fac
    dup
    invokespecial Fac/<init>()V
    ldc 10
    invokevirtual Fac/ComputeFac(I)I
    invokevirtual java/io/PrintStream/println(I)V
    return
.end method
```

Fac.j

```
.class public Fac
.super java/lang/Object
.method public <init>()V
    .limit stack 1
    .limit locals 1
    aload_0
    invokespecial java/lang/Object/<init>()V
    return
.end method
.method public ComputeFac(I)I
```

```

        .limit stack 20
        .limit locals 3
        iload 1
        ldc 1
        if_icmplt L2
        iconst_0
        goto L3
L2:
        iconst_1
L3:
        ifeq L0
        ldc 1
        istore 2
        goto L1
L0:
        iload 1
        aload_0
        iload 1
        ldc 1
        isub
        invokevirtual Fac/ComputeFac(I)I
        imul
        istore 2
L1:
        iload 2
        ireturn
    .end method

```

Salida:

```

run-compiled:
  [java] 3628800
BUILD SUCCESSFUL
Total time: 1 second

```

```
<terminated> Compilador2026 build.xml [Ant Build] C:\Users\DELL\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32
[java]      System.out.println(new Fac().ComputeFac(10));
[java]    }
[java]  }
[java] class Fac {
[java]   public int ComputeFac (int num) {
[java]     int num_aux;
[java]     if ((num < 1))
[java]       num_aux = 1;
[java]     else num_aux = (num * this.ComputeFac((num - 1)));
[java]     return num_aux;
[java]   }
[java] }
[java] Analisis semantico: OK
[java] Archivos .j generados en /output
run-jasmin:
[apply] Generated: output\BBS.class
[apply] Generated: output\BubbleSort.class
[apply] Generated: output\Fac.class
[apply] Generated: output\Factorial.class
[apply] Generated: output\QS.class
[apply] Generated: output\QuickSort.class
run-compiled:
[java] 3628800
BUILD SUCCESSFUL
Total time: 1 second
```